CIS 4020 · DATA STRUCTURES

My Notes: Hashing

Hash functions, collision resolution, and counting probes

A note before you begin. A word before you read. These are my notes from years of teaching this unit, cleaned up so they are readable. While C/C++ was my first language, I am a bit rusty because I have taught Java for years now. For these notes, I have been using AI to help type these up, check for issues, and fill in explanations, which means a wrong or imprecise statement can slip in and survive longer than it should, and I do not fully trust the AI, so I am still editing. They are not meant to be a study guide, they are not gospel, and not everything here shows up in a lecture (and some lecture items are not here). If you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing students and me a favor, and you will understand it better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. We are a two-language course, thus I will do my best to show everything in Java (CIS) and C/C++ (CS).

1. The idea: calculate the address instead of searching for it

A hash table stores data in a value-oriented linear structure built for fast search. The path from a key to its location has three parts:

$$\text{search key } x \quad \rightarrow \quad \text{address calculator } h(x) \quad \rightarrow \quad \text{table location}$$

The table location is usually an array element, or a node in a linked list hanging off an array element. Three terms come up constantly:

- **Hashing** is the act of converting x into its hash value.
- The **hash function** $h(x)$ is the address calculator.
- The **hash table** is the data structure that stores the items.

When things go well, operations are $O(1)$: we compute $h(x)$, jump to that slot, and the item is there. We do not have to look through other items to find it. The rest of this unit is about what happens when things do not go perfectly, and how much that costs.

2. A first example by hand

This is the example I work on the board. The table has 10 slots, so the hash function is

$$h(x) = x \bmod 10,$$

where 10 is the table size. We insert the keys 100, 92, 54, 71, 23, 36, 40, 45, in that order. When a slot is already full, we step forward one slot at a time until we find an empty one. That is called *linear probing*, and we will name it properly in Section 4.

Table 1: Inserting eight keys into a 10-slot table with $h(x) = x \bmod 10$ and linear probing.

Key x	$h(x)$	Final slot	What happened
100	0	0	Slot 0 empty, placed directly
92	2	2	Slot 2 empty, placed directly
54	4	4	Slot 4 empty, placed directly
71	1	1	Slot 1 empty, placed directly
23	3	3	Slot 3 empty, placed directly
36	6	6	Slot 6 empty, placed directly
40	0	5	Slots 0, 1, 2, 3, 4 full; placed in slot 5
45	5	7	Slots 5, 6 full; placed in slot 7

The final table holds 100, 71, 92, 23, 54, 40, 36, 45 in slots 0 through 7, and slots 8 and 9 are empty. Notice two things. First, the six keys that did not collide each took one probe to place, and they will take one probe to find later. Second, 40 needed six probes, and its arrival is the reason 45 then needed three. Collisions create more collisions. That observation is the seed of everything in Section 5.

3. What makes a good hash function

A **collision** happens when $h(x)$ maps more than one item to the same location. We cannot avoid collisions entirely in general, but a good hash function keeps them rare. A good hash function:

- **is easy and fast to compute.** $O(1)$ work is easy to achieve and is what we want, since we compute it on every operation.
- **distributes items uniformly throughout the table.** This is what avoids collisions. Think about whether your keys are random or follow a pattern; student IDs that all start with the same digits, for example, are far from random.
- **involves the entire search key.** If h looks at only part of the key, keys that differ only in the ignored part all collide.

A **perfect hash function** produces no collisions at all. That is only possible when every search key is known in advance, such as the reserved words of a programming language.

Common techniques for integer keys

Hash functions that work on positive integers are common and are usually enough, because other kinds of keys can be turned into integers first. Using the key 001364825:

Table 2: Integer hashing techniques applied to the key 001364825.

Technique	Example	Result
Selecting digits	The 4th digit and the last digit: 3 and 5	35
Folding (digits)	Add every digit: $0+0+1+3+6+4+8+2+5$	29
Folding (groups)	Add groups of three: $001 + 364 + 825$	1190
Modulus	001364825 mod 101	12

Selecting digits is fast but ignores most of the key, which breaks the third rule above. Folding uses the whole key, but the result can be larger than the table (1190 would not fit in a 101-slot table), so folding is often followed by a modulus. The modulus is the workhorse: x mod tableSize always lands in range.

Use a prime table size. With modulus hashing, the table size should be a prime number. A size like 10 or 40 shares factors with many common keys, so those keys pile into the same few slots. A prime size spreads them out. You will see a dramatic version of this problem in Section 7.

String keys

To hash a string, convert it to an integer, then apply one of the integer techniques. Common conversions:

- Use the ASCII (or Unicode) value of each character.
- Use the position in the alphabet, A = 1 through Z = 26.
- Write each position in binary and concatenate. N is position 14, which is `01110` in five bits.

For example, the key CAT is C = 3 = `00011`, A = 1 = `00001`, and T = 20 = `10100`. Concatenated, that is `000110000110100`, which is 3124 in decimal, and 3124 mod 101 = 94. Simply adding the character values would also work but is a weak choice: CAT, ACT, and TAC would all collide, because adding ignores the order of the characters.

4. Resolving collisions

The simplest fix, making the table bigger, is not always practical. The real techniques fall into two families.

Open addressing: probe for another open slot

In open addressing, every item lives in the array itself. When the home slot $h(x)$ is taken, we *probe* a sequence of other slots until we find an empty one. The same sequence is followed when searching, so a search for x ends when it finds x or reaches an empty slot. For probe number $i = 0, 1, 2, \ldots$:

Table 3: Probe sequences for open addressing. All results are taken modulo the table size.

Technique	Slot checked on probe i	Sequence of offsets
Linear probing	$h(x) + i$	$0, 1, 2, 3, 4, \ldots$
Quadratic probing	$h(x) + i^2$	$0, 1, 4, 9, 16, \ldots$
Double hashing	$h_1(x) + i \cdot h_2(x)$	$0, h_2, 2h_2, 3h_2, \ldots$

Linear probing is a sequential search starting at the home slot. Its weakness is **primary clustering**: occupied slots form long runs near the hash positions, and any key that hashes anywhere into a run has to walk to its end, which makes the run longer still. The 40 and 45 in Table 1 are exactly this.

Quadratic probing jumps by the squares of i, so it moves away from a crowded area quickly. Note that the offsets are squares (1, 4, 9, 16), not powers of 2. Quadratic probing eliminates primary clustering, but keys with the same home slot still follow the identical sequence, which is called **secondary clustering**: clusters "away" from the hash positions rather than right at them.

Double hashing uses a second hash function $h_2(x)$ to choose the step size, so two keys with the same home slot usually step differently. This removes both kinds of clustering in practice. $h_2(x)$ must never be 0 (or the probe would never move), and the step size and table size should have no common factor, which a prime table size guarantees. As long as h_2 is chosen well, double hashing largely avoids both primary and secondary clustering.

A short example of double hashing, with a table size of 11, $h_1(x) = x \bmod 11$, and $h_2(x) = 7 - (x \bmod 7)$:

- 58: $h_1 = 3$. Slot 3 is empty, so 58 goes there.
- 14: $h_1 = 3$, which is taken. $h_2 = 7 - 0 = 7$, so try $3 + 7 = 10$. Empty, so 14 goes in slot 10.
- 91: $h_1 = 3$, taken. $h_2 = 7$, so try 10, also taken. Try $3 + 14 = 17$, and $17 \bmod 11 = 6$. Empty, so 91 goes in slot 6.
- 25: $h_1 = 3$, taken. $h_2 = 7 - 4 = 3$, so try 6, taken. Try $3 + 6 = 9$. Empty, so 25 goes in slot 9.

Notice that 14, 91, and 25 all start at slot 3, but 25 steps by 3 instead of 7, so it does not follow the other two. That is the point of the second hash function.

Deleting under open addressing

Deleting from an open-addressed table is not as simple as emptying the slot. If 92 is removed from Table 1 by setting slot 2 to empty, a later search for 40 would stop at slot 2 and wrongly report that 40 is missing. The fix is to mark a removed slot as *deleted*, not empty. A search keeps going past a deleted slot; an insertion is allowed to reuse it. So each slot is in one of three states: empty, occupied, or deleted.

Separate chaining: buckets

In **separate chaining**, each array element holds a linked list, often called a bucket, of every item that hashed there. A collision simply adds another node to that list. There is no probing into other slots and no clustering, deletion is an ordinary linked list removal, and the table can even hold more

items than it has slots ($\alpha > 1$). The cost is the extra memory for the links, and a long chain is just as slow as any linked list.

5. Efficiency and the load factor

The **load factor** measures how full the table is:

$$\alpha = \frac{\text{current number of items}}{\text{table size}}.$$

For open addressing, α is between 0 and 1. The expected number of probes depends almost entirely on α, not on the number of items. A table of 1,000 slots holding 500 items performs about the same as a table of 1,000,000 slots holding 500,000 items. That is what makes hashing feel like $O(1)$, and it is why the textbook's goal is to keep α at or below 2/3 for open addressing.

The textbook gives the average number of probes for each technique.

Table 4: Average number of probes for successful and unsuccessful searches, as functions of the load factor α.

Technique	Successful search	Unsuccessful search
Linear probing	$\frac{1}{2}\left[1 + \frac{1}{1-\alpha}\right]$	$\frac{1}{2}\left[1 + \frac{1}{(1-\alpha)^2}\right]$
Quadratic probing and double hashing	$\frac{-\ln(1-\alpha)}{\alpha}$	$\frac{1}{1-\alpha}$
Separate chaining	$1 + \frac{\alpha}{2}$	α

Formulas are easier to feel with numbers in them.

Table 5: Values of the efficiency formulas at several load factors. "Lin." is linear probing, "Quad." is quadratic probing or double hashing, "Succ." is a successful search, and "Unsucc." is an unsuccessful search.

α	Lin. succ.	Lin. unsucc.	Quad. succ.	Quad. unsucc.	Chain succ.	Chain unsucc.
0.25	1.17	1.39	1.15	1.33	1.13	0.25
0.50	1.50	2.50	1.39	2.00	1.25	0.50
0.67	2.02	5.09	1.65	3.03	1.33	0.67
0.75	2.50	8.50	1.85	4.00	1.38	0.75
0.90	5.50	50.50	2.56	10.00	1.45	0.90

Three patterns are worth remembering:

1. **Unsuccessful searches cost more than successful ones** for open addressing. A successful search stops as soon as it finds the key, often partway through a cluster. An unsuccessful search has to reach an empty slot, so it always walks the whole cluster.
2. **Linear probing falls apart fastest as the table fills.** Going from $\alpha = 0.75$ to $\alpha = 0.9$ takes its unsuccessful search from 8.5 probes to 50.5. That is primary clustering showing up in the math.
3. **Separate chaining barely notices.** Its unsuccessful value is just α, the average chain length, because an unsuccessful search walks one whole chain (an empty chain costs nothing to walk).

6. The spreadsheet demonstration

In class we test the formulas against an actual table. The spreadsheet inserts 30 keys into a 41-slot table using $h(x) = x \bmod 41$, so when it is done,

$$\alpha = \frac{30}{41} \approx 0.73.$$

The table size is 41 because it is prime. Section 7 uses a second workbook to show what can go wrong without that. There are three sheets, one each for linear probing, quadratic probing, and double hashing with $h_2(x) = 7 - (x \bmod 7)$. Each sheet has the same steps:

1. Insert the 30 keys in order, resolving collisions with that sheet's technique.
2. For each of the 30 keys in the table, count the probes it takes to find it.
3. For each of 30 keys *not* in the table, count the probes it takes to decide it is missing.
4. Average each column and compare it with the formula at $\alpha = 30/41$.

Each sheet also has a probe sequence helper: type in a key, and it lists the slots that key would check, what is in each one, and where the search stops.

How to count a probe

Every table location you check counts as one probe, including the first one. A successful search counts the slot where the key is finally found. An unsuccessful search counts the empty slot that ends it. In the linear probing sheet, for example, searching for 11 starts at slot 11 and checks slots 11, 12, 13, 14, 15, and 16 (all occupied by other keys), then slot 17, which is empty: 7 probes.

What the counts show

Table 6: Hand-counted averages from the spreadsheet compared with the formulas at $\alpha = 30/41 \approx 0.73$ (30 keys, 41 slots).

Measurement	Counted average	Formula
Linear, successful	1.47	2.36
Linear, unsuccessful	4.30	7.45
Quadratic, successful	1.73	1.80
Quadratic, unsuccessful	3.57	3.73
Double hashing, successful	1.53	1.80
Double hashing, unsuccessful	4.57	3.73

The counts do not match the formulas, and that is fine. The formulas describe the *expected* value over all possible sets of keys with perfectly random hash values. Our spreadsheet is one set of 30 small keys, chosen by hand. What the counts do confirm:

- Unsuccessful searches cost more than successful ones, on all three sheets.
- The rough size of the numbers matches the formulas well enough that the formulas stop being abstract.

Some results go against the formulas. Quadratic probing does worse than linear probing on successful searches (1.73 versus 1.47), and double hashing does worse than linear probing on unsuccessful searches (4.57 versus 4.30). The reason is the keys. Many of them are small and close together, such as 41, 42, 44, and 46, which is far from random. The formulas assume random keys, and with only 30 hand-picked ones, a few unlucky collisions can move an average noticeably. This is why the spreadsheet is a rough estimate, not a proof.

7. What goes wrong with 40 slots

Open the workbook `Hashing_TableSize40.xlsx`. It uses the same 30 keys as the 41-slot workbook, but with 40 slots and $h(x) = x \bmod 40$. It is already filled in, so you can study it rather than build it. On its Quadratic sheet, the unsuccessful search for 96 never ends: it never finds an empty slot, and it would loop forever. The sheet marks that search "Never ends" and leaves it out of the average.

Here is why. The home slot is $96 \bmod 40 = 16$, and quadratic probing checks $(16 + i^2) \bmod 40$. But $i^2 \bmod 40$ can only ever be one of nine values:

$$i^2 \bmod 40 \in \{0, 1, 4, 9, 16, 20, 24, 25, 36\}.$$

So the search starting at slot 16 can only ever visit nine slots: 16, 17, 20, 25, 32, 36, 0, 1, and 12. In that table, all nine are occupied. The other 31 slots, including the ten empty ones, are unreachable from slot 16. You can watch this happen: the probe sequence helper on the Quadratic sheet is already set to trace 96, and every row it lists says Occupied while the slots checked cycle through the same nine values. Trace 96 on the Linear sheet for comparison, and the search stops after a few probes.

Why prime table sizes matter. When the table size is a prime number p, the first $\lceil p/2 \rceil$ quadratic probes all land on different slots. That guarantees quadratic probing will find an empty slot whenever the table is less than half full. With $p = 41$, $i^2 \bmod 41$ takes 21 different values instead of 9, and in our 41-slot spreadsheet every insert and search finishes. Note that even a prime size only guarantees success below $\alpha = 1/2$, so a quadratic probing implementation should still stop after a fixed number of probes.

What this means in code

Every open-addressing loop needs a stopping rule besides "found it" and "found an empty slot." The simplest rule is to stop after as many probes as there are slots. Here is a linear probing search in both languages, with that bound in place. For quadratic probing, change one line, so that `index = (home + i * i) % size`.

Java (CIS): linear probing search

```java
// Returns the index where key is stored, or -1 if it is not in the table.
// A null slot means that slot has never been used.
public static int find(Integer[] table, int key) {
    int size = table.length;
    int home = key % size;                  // h(x)
    for (int i = 0; i < size; i++) {        // never more than size probes
        int index = (home + i) % size;      // linear: h(x) + i
        if (table[index] == null) {
            return -1;                      // empty slot: key is not here
        }
        if (table[index] == key) {
            return index;                   // found it
        }
    }
    return -1;                              // checked every slot
}
```

C/C++ (CS): linear probing search

```c
const int EMPTY = -1;   // keys are nonnegative, so -1 marks an unused slot

// Returns the index where key is stored, or -1 if it is not in the table.
int find(const int table[], int size, int key) {
    int home = key % size;                  // h(x)
    for (int i = 0; i < size; i++) {        // never more than size probes
        int index = (home + i) % size;      // linear: h(x) + i
        if (table[index] == EMPTY) {
            return -1;                      // empty slot: key is not here
        }
        if (table[index] == key) {
            return index;                   // found it
        }
    }
    return -1;                              // checked every slot
}
```

These versions do not handle the deleted state from the previous section; adding it means one more check that skips a deleted slot instead of stopping there.

8. Common pitfalls

- **Forgetting to count the first probe.** Checking the home slot is probe 1, not probe 0.
- **Stopping an unsuccessful search at a deleted slot.** Only an empty slot ends a search.
- **Using squares as powers of 2.** Quadratic probing offsets are 1, 4, 9, 16, 25, not 1, 2, 4, 8, 16.
- **Picking a table size like 10, 40, or 100.** Use a prime. It matters most for quadratic probing and double hashing.

- **Letting α creep up.** Past about 2/3, open addressing gets expensive quickly. The usual fix is to allocate a larger (prime) table and reinsert every item, called rehashing. Each item must be rehashed, because $h(x)$ changes with the table size.
- **Expecting order.** Hash tables are poor at listing items in sorted order or finding a range of keys. Traversing a hash table visits items in scattered order.

9. Textbook sections

- **Java (CIS):** Prichard and Carrano, *Data Abstraction and Problem Solving with Java: Walls and Mirrors*, 3rd ed., Chapter 13, "Advanced Implementations of Tables," Section 13.2, "Hashing." Its subsections cover hash functions, resolving collisions, the efficiency of hashing, what makes a good hash function, why table traversal is inefficient under hashing, and the Java Collections Framework `Hashtable` class.
- **C/C++ (CS):** Carrano and Henry, *Secure Data Structures and Algorithms with C++: Walls and Mirrors*, 8th ed., Chapter 24, "Hashing as a Map Implementation," with Chapter 23, "Maps and Their Implementations," for the ADT itself.